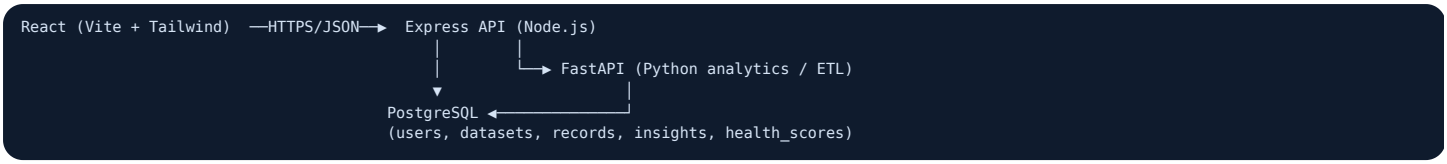
Business Intelligence & Decision Support Platform

Ye document aapko step-by-step batata hai ki frontend aur backend ko production-style me kaise banana hai — pehle setup, phir features, phir integration aur deployment.

Architecture



Tech stack & ownership

Layer	Technology	Owner
Frontend	React.js, Tailwind CSS, React Router, Axios, Recharts	Nirmal
Backend	Node.js, Express.js, Multer, Zod/Joi	Priyanshu
Database	PostgreSQL (pg / Prisma optional)	Priyanshu
Analytics API	FastAPI, Python, Pandas	Tanishq + Priyanshu
Auth	JWT, bcrypt	Priyanshu
Version control	Git + GitHub (feature branches, PR review)	All

Kaam ka order (roadmap)

1. **Week 1:** Backend server + DB connection | Frontend setup + routing + common components
2. **Week 2:** Auth APIs + Auth screens → integration
3. **Week 3:** Dataset upload API + Upload page → FastAPI ETL integration
4. **Week 4:** Dashboard APIs + Dashboard UI (dummy → real data)
5. **Week 5:** Health score + Insights (rules) + Insights UI
6. **Week 6:** Testing, bug fixes, README, deployment, demo

Git workflow (real-world)

- Branches: `main` (stable) → `develop` → `feature/auth-ui`, `feature/upload-api`
- Commit style: `feat: add login page`, `fix: token refresh bug`
- Har feature ka Pull Request banao, kam se kam 1 member review kare, phir merge.
- `.env` kabhi commit nahi hoga — sirf `.env.example`.

> **Golden rule:** Pehle API contract (request/response format) teeno members finalize karein, phir frontend aur backend parallel me kaam kar sakte hain.

Frontend Setup (Phase 1)

React + Vite + Tailwind ka clean, scalable base.

Step 1 — Project create

```
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm install react-router-dom axios recharts lucide-react
npm install -D tailwindcss @tailwindcss/vite
```

Step 2 — Tailwind (v4) configure

```
// vite.config.js
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'
export default defineConfig({ plugins: [react(), tailwindcss()] })

/* src/index.css */
@import "tailwindcss";
```

Step 3 — Folder structure

```
src/
├── components/   Navbar, Sidebar, Button, Card, Loader, Table, EmptyState, Toast
├── pages/        Login, Register, Dashboard, UploadDataset, DatasetHistory,
│                 Insights, InsightDetail, Profile, Reports, Help, NotFound
├── services/     api.js, authService.js, datasetService.js, dashboardService.js
├── context/      AuthContext.jsx
├── hooks/        useFetch.js, useAuth.js
├── routes/       ProtectedRoute.jsx
├── utils/        validators.js, formatters.js
├── App.jsx
└── main.jsx
```

Step 4 — Axios instance (auto token + error handling)

```
// src/services/api.js
import axios from 'axios'
const api = axios.create({ baseURL: import.meta.env.VITE_API_URL, timeout: 30000 })

api.interceptors.request.use(cfg => {
  const token = localStorage.getItem('token')
  if (token) cfg.headers.Authorization = `Bearer ${token}`
  return cfg
})
api.interceptors.response.use(r => r, err => {
  if (err.response?.status === 401) {
    localStorage.removeItem('token'); window.location.href = '/login'
  }
  return Promise.reject(err)
})
export default api
```

Step 5 — Routing + Protected routes

```
// routes/ProtectedRoute.jsx
import { Navigate, Outlet } from 'react-router-dom'
import { useAuth } from '../context/AuthContext'
export default function ProtectedRoute() {
  const { user, loading } = useAuth()
  if (loading) return <Loader />
  return user ? <Outlet /> : <Navigate to="/login" replace />
}

// App.jsx
<Routes>
  <Route path="/login" element={<Login/>} />
  <Route path="/register" element={<Register/>} />
  <Route element={<ProtectedRoute/>}>
    <Route element={<Layout/>}>   {/* Navbar + Sidebar */}
      <Route path="/" element={<Dashboard/>} />
      <Route path="/upload" element={<UploadDataset/>} />
      <Route path="/datasets" element={<DatasetHistory/>} />
      <Route path="/insights" element={<Insights/>} />
      <Route path="/insights/:id" element={<InsightDetail/>} />
      <Route path="/profile" element={<Profile/>} />
    </Route>
  </Route>
  <Route path="*" element={<NotFound/>} />
</Routes>
```

> **.env.example (frontend):** `VITE_API_URL=http://localhost:5000/api`

Common components checklist

☐ Button (variants + loading state) ☐ Card (KPI card) ☐ Loader / Skeleton ☐ Input with error text ☐ Toast / Alert
☐ Modal (delete confirm) ☐ EmptyState

Frontend Pages & Features (Phase 2–6)

Authentication screens

- **Login/Register:** email format check, password min 8 chars (1 uppercase + 1 number), confirm password, show/hide toggle
- Button loading state, backend error message display (invalid credentials / email already exists)
- **AuthContext:** `login()`, `register()`, `logout()`, page refresh par `/auth/profile` se user restore

```
// context/AuthContext.jsx (core)
useEffect(() => {
  const token = localStorage.getItem('token')
  if (!token) return setLoading(false)
  api.get('/auth/profile').then(r => setUser(r.data.data)).catch(() => localStorage.removeItem('token')).finally(() => setLoading(false))
}, [])
const login = async (email, password) => {
  const { data } = await api.post('/auth/login', { email, password })
  localStorage.setItem('token', data.data.token); setUser(data.data.user)
}
const logout = () => { localStorage.removeItem('token'); setUser(null) }
```

Dashboard layout

Row 1 — KPI cards

Total Sales, Total Profit, Total Orders, Total Quantity, Avg Order Value, Business Health Score

Row 2 — Trends

Sales trend (LineChart), Profit trend (AreaChart)

Row 3 — Breakdown

Category performance (BarChart), Region performance (BarChart / Pie)

Row 4

Recent insights list + dataset selector dropdown

Abhi dummy data use karo: `src/mock/dashboard.js` me backend jaisi hi response shape rakho, taaki baad me sirf service function badalna pade.

Upload page

- Drag & drop + browse button; sirf `.csv`, `.xlsx`, `.xls`
- Size limit (e.g. 10 MB) client-side check; empty file reject
- Progress bar via Axios `onUploadProgress`
- Success → dataset summary (rows, columns, status) + first 10 rows preview table
- Upload history table: file name, type, status badge (Uploaded / Processed / Failed), date, delete action

```
const formData = new FormData(); formData.append('file', file)
await api.post('/datasets/upload', formData, {
  headers: { 'Content-Type': 'multipart/form-data' },
  onUploadProgress: e => setProgress(Math.round((e.loaded * 100) / e.total))
})
```

Insights UI

- Insight card: title, category chip, **severity badge** (Low = green, Medium = amber, High = red), description
- Recommendation card alag se highlight karo
- Filters: category + severity; search optional
- Insight detail page + "Business Health Score kaise calculate hota hai" explanation panel

Extra pages & quality

- Profile, Dataset history, Reports (export PDF/CSV later), Help/About, 404 & error boundary
- Har page par 3 states: **loading** (skeleton), **error** (retry button), **empty** (call-to-action)
- Responsive: sidebar mobile me drawer; test 360px, 768px, 1280px

> **Dhyan rakho:** Token localStorage me simple hai (project ke liye theek). Production me httpOnly cookie zyada secure hoti hai — report me ye trade-off mention karo.

Backend Setup + Database (Phase 1–2)

Step 1 — Project setup

```
mkdir backend && cd backend && npm init -y
npm i express pg dotenv cors helmet morgan bcrypt jsonwebtoken multer zod express-rate-limit axios form-data
npm i -D nodemon
```

Step 2 — Structure

```
backend/
├── src/
│   ├── config/        database.js, env.js
│   ├── controllers/   auth, dataset, dashboard, insight
│   ├── routes/        authRoutes, datasetRoutes, dashboardRoutes, insightRoutes
│   ├── middleware/    authMiddleware, uploadMiddleware, validate, errorHandler
│   ├── services/      analyticsService.js (FastAPI calls)
│   ├── utils/         ApiError.js, asyncHandler.js
│   ├── app.js
│   └── server.js
├── db/schema.sql
├── uploads/
├── .env / .env.example
└── package.json
```

Step 3 — Core files

```
// config/database.js
const { Pool } = require('pg')
module.exports = new Pool({ connectionString: process.env.DATABASE_URL })

// app.js
const express = require('express'), cors = require('cors'), helmet = require('helmet'), morgan = require('morgan')
const app = express()
app.use(helmet()); app.use(morgan('dev'))
app.use(cors({ origin: process.env.CLIENT_URL, credentials: true }))
app.use(express.json({ limit: '1mb' }))
app.use('/api/auth', require('./routes/authRoutes'))
app.use('/api/datasets', require('./routes/datasetRoutes'))
app.use('/api/dashboard', require('./routes/dashboardRoutes'))
app.use('/api/insights', require('./routes/insightRoutes'))
app.use(require('./middleware/errorHandler'))
module.exports = app

// middleware/errorHandler.js
module.exports = (err, req, res, next) => {
  res.status(err.status || 500).json({ success: false, message: err.message || 'Server error' })
}
```

.env.example: `PORT=5000` , `DATABASE_URL=postgres://user:pass@localhost:5432/bidsp` , `JWT_SECRET=change_me` , `JWT_EXPIRES_IN=1d` , `CLIENT_URL=http://localhost:5173` , `ANALYTICS_URL=http://localhost:8000`

Step 4 — Database schema (db/schema.sql)

```
CREATE TABLE users (
  id SERIAL PRIMARY KEY, name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL, password_hash TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW());

CREATE TABLE datasets (
  id SERIAL PRIMARY KEY, user_id INT REFERENCES users(id) ON DELETE CASCADE,
  file_name VARCHAR(255) NOT NULL, file_type VARCHAR(10) NOT NULL,
  file_path TEXT, row_count INT, status VARCHAR(20) DEFAULT 'uploaded'
    CHECK (status IN ('uploaded','processing','processed','failed')),
  uploaded_at TIMESTAMPTZ DEFAULT NOW());

CREATE TABLE dataset_records (            -- columns final dataset ke hisab se
  id BIGSERIAL PRIMARY KEY, dataset_id INT REFERENCES datasets(id) ON DELETE CASCADE,
  order_id VARCHAR(50), order_date DATE, category VARCHAR(100), region VARCHAR(100),
  ship_mode VARCHAR(50), sales NUMERIC(12,2), quantity INT, discount NUMERIC(5,2), profit NUMERIC(12,2));
CREATE INDEX idx_records_dataset ON dataset_records(dataset_id);
CREATE INDEX idx_records_date ON dataset_records(dataset_id, order_date);

CREATE TABLE insights (
  id SERIAL PRIMARY KEY, dataset_id INT REFERENCES datasets(id) ON DELETE CASCADE,
  category VARCHAR(50), title VARCHAR(200), description TEXT,
  severity VARCHAR(10) CHECK (severity IN ('low','medium','high')),
  recommendation TEXT, created_at TIMESTAMPTZ DEFAULT NOW());

CREATE TABLE health_scores (
  id SERIAL PRIMARY KEY, dataset_id INT REFERENCES datasets(id) ON DELETE CASCADE,
  score NUMERIC(5,2), score_category VARCHAR(30), breakdown JSONB,
  calculated_at TIMESTAMPTZ DEFAULT NOW());
```

Auth & Dataset APIs (Phase 3–4)

Standard API response format (sabhi APIs me same)

```
// success
{ "success": true, "message": "Login successful", "data": { ... } }
// error
{ "success": false, "message": "Email already registered", "errors": [ ... ] }
```

Auth endpoints

Method	Endpoint	Details
POST	`/api/auth/register`	Validate → check duplicate email (409) → bcrypt hash → insert
POST	`/api/auth/login`	Compare hash → sign JWT → return token + user
GET	`/api/auth/profile`	Protected; return current user (password_hash kabhi nahi)
POST	`/api/auth/logout`	Stateless JWT: client token delete karta hai

```
// controllers/authController.js (register + login)
const bcrypt = require('bcrypt'), jwt = require('jsonwebtoken'), db = require('../config/database')
exports.register = async (req, res) => {
  const { name, email, password } = req.body            // zod se validated
  const exists = await db.query('SELECT 1 FROM users WHERE email=$1', [email])
  if (exists.rowCount) return res.status(409).json({ success:false, message:'Email already registered' })
  const hash = await bcrypt.hash(password, 12)
  const { rows } = await db.query(
    'INSERT INTO users(name,email,password_hash) VALUES($1,$2,$3) RETURNING id,name,email', [name,email,hash])
  res.status(201).json({ success:true, message:'Registered', data: rows[0] })
}
exports.login = async (req, res) => {
  const { email, password } = req.body
  const { rows } = await db.query('SELECT * FROM users WHERE email=$1', [email])
  if (!rows[0] || !(await bcrypt.compare(password, rows[0].password_hash)))
    return res.status(401).json({ success:false, message:'Invalid email or password' })
  const token = jwt.sign({ id: rows[0].id }, process.env.JWT_SECRET, { expiresIn: process.env.JWT_EXPIRES_IN })
  res.json({ success:true, data:{ token, user:{ id:rows[0].id, name:rows[0].name, email } } })
}

// middleware/authMiddleware.js
module.exports = (req, res, next) => {
  const h = req.headers.authorization
  if (!h?.startsWith('Bearer ')) return res.status(401).json({ success:false, message:'Not authorized' })
  try { req.user = jwt.verify(h.split(' ')[1], process.env.JWT_SECRET); next() }
  catch { res.status(401).json({ success:false, message:'Token invalid or expired' }) }
}
```

Login route par `express-rate-limit` lagao (e.g. 10 attempts / 15 min) — brute force se bachne ke liye.

Dataset endpoints

Method	Endpoint	Purpose
POST	`/api/datasets/upload`	Multipart upload, validate, ETL trigger
GET	`/api/datasets`	Logged-in user ke datasets (history)
GET	`/api/datasets/:id`	Metadata + status + summary
GET	`/api/datasets/:id/preview`	First 10–20 rows
DELETE	`/api/datasets/:id`	Ownership check → delete file + DB rows

```
// middleware/uploadMiddleware.js
const multer = require('multer'), path = require('path')
const allowed = ['.csv', '.xlsx', '.xls']
module.exports = multer({
  storage: multer.diskStorage({
    destination: 'uploads/',
    filename: (req, f, cb) => cb(null, `${Date.now()}-${Math.round(Math.random()*1e9)}${path.extname(f.originalname)}`)
  }),
  limits: { fileSize: 10 * 1024 * 1024 },
  fileFilter: (req, f, cb) => allowed.includes(path.extname(f.originalname).toLowerCase())
    ? cb(null, true) : cb(new Error('Only CSV or Excel files allowed'))
})
```

Upload flow: Frontend → Express (validate + save) → dataset row `status='processing'` → FastAPI clean/validate → records PostgreSQL me → `status='processed'` (ya `'failed'`) → frontend status poll karta hai.

Security checklist: ownership check har dataset query me (`WHERE id=$1 AND user_id=$2`) • parameterized queries (SQL injection se bachav) • empty file reject • error me stack trace na bhejo.

Dashboard, Analytics, Health Score & Insights (Phase 5–8)

Dashboard APIs (SQL aggregation)

Endpoint	Returns
`GET /api/dashboard/:datasetId/summary`	total_sales, total_profit, total_orders, total_quantity, avg_order_value, health_score
`.../sales-trend`	monthly [{ month, sales }]
`.../profit-trend`	monthly [{ month, profit }]
`.../category-performance`	[{ category, sales, profit, margin }]
`.../region-performance`	[{ region, sales, profit }]
`.../shipping-analysis`	[{ ship_mode, orders, avg_days? }]

```
-- summary
SELECT SUM(sales) total_sales, SUM(profit) total_profit,
       COUNT(DISTINCT order_id) total_orders, SUM(quantity) total_quantity,
       SUM(sales)/NULLIF(COUNT(DISTINCT order_id),0) avg_order_value
FROM dataset_records WHERE dataset_id = $1;

-- sales trend
SELECT TO_CHAR(DATE_TRUNC('month', order_date),'YYYY-MM') month, SUM(sales) sales
FROM dataset_records WHERE dataset_id=$1 GROUP BY 1 ORDER BY 1;
```

Har dashboard route me pehle check karo ki dataset current user ka hai; middleware `checkDatasetOwner` banao.

FastAPI integration (services/analyticsService.js)

```
const axios = require('axios')
const py = axios.create({ baseURL: process.env.ANALYTICS_URL, timeout: 120000 })
exports.processDataset = async (datasetId, filePath) => {
  await py.post('/analytics/validate', { dataset_id: datasetId, file_path: filePath })
  await py.post('/analytics/clean', { dataset_id: datasetId })
  await py.post('/analytics/analyze', { dataset_id: datasetId })
  await py.post('/analytics/health-score', { dataset_id: datasetId })
  await py.post('/analytics/insights', { dataset_id: datasetId })
}
```

FastAPI endpoint	Kaam
POST /analytics/validate	Required columns, data types, empty rows check
POST /analytics/clean	Duplicates, nulls, format fix → DB insert
POST /analytics/analyze	KPI + aggregate calculation
POST /analytics/health-score	Documented formula se score
POST /analytics/insights	Rule engine → insights table
GET /analytics/status/:datasetId	Processing status

Business Health Score (reproducible)

Formula team finalize karegi aur report me likhegi. Example structure:

```
Health Score = 0.30*Sales + 0.30*Profitability + 0.20*Orders + 0.20*Regional
Har component 0–100 (thresholds document me define)
Category: 80–100 Excellent | 60–79 Good | 40–59 Average | <40 Needs Attention
```

Breakdown JSON `health_scores.breakdown` me save karo taaki frontend explanation dikha sake.

Rule-based insights

```
// rules example (thresholds config file me rakho, hardcode nahi)
{ id:'LOW_MARGIN', category:'Profit', check: m => m.profit_margin < T.MIN_MARGIN,
  severity: m => m.profit_margin < T.CRITICAL_MARGIN ? 'high' : 'medium',
  title:'Low Profitability',
  recommendation:'Review low-margin products and evaluate discount strategies.' }
```

- Rule groups: Sales, Profit, Discount, Category, Regional
- Insights API: `GET /api/insights?datasetId=&category=&severity=` aur `GET /api/insights/:id`
- Rerun karne par purane insights delete karke naye insert (duplicate na bane)

Integration, Testing & Delivery

Integration sequence (follow this order)

1. Backend server + DB connection (health check `GET /api/health`)
2. Frontend UI + routing (dummy data)
3. Auth integration (register → login → profile → protected routes)
4. Dataset upload integration (progress, status polling)
5. FastAPI analytics integration
6. Dashboard integration (dummy service → real API)
7. Insights + Health Score integration
8. Final testing + deployment

Tip: Postman/Thunder Client collection banao aur share karo — frontend developer backend ready hone se pehle bhi contract dekh sake. Swagger docs bonus me.

Testing checklist

Auth

- ☐ Register valid / duplicate email
- ☐ Login wrong password
- ☐ Expired token → redirect
- ☐ Logout

Upload

- ☐ Invalid type (.pdf, .exe)
- ☐ Empty file
- ☐ File > 10 MB
- ☐ Duplicate upload
- ☐ Missing columns

Data

- ☐ KPI = manual Excel calculation
- ☐ Rule-based insights correct
- ☐ Health score reproducible

UI / API

- ☐ API 400/401/404/500 handling
- ☐ Mobile + desktop
- ☐ Empty / loading / error states

Deployment (recommended free/cheap stack)

Part	Platform	Note
Frontend	Vercel / Netlify	`npm run build`, set `VITE_API_URL`
Backend	Render / Railway	Env vars dashboard me; start: `node src/server.js`
FastAPI	Render / Railway	`uvicorn main:app --host 0.0.0.0`
Database	Neon / Supabase / Railway PG	SSL enable, backup on

Production me CORS sirf frontend domain ke liye, strong `JWT_SECRET`, HTTPS only, uploads ke liye cloud storage (S3/Cloudinary) consider karo kyunki Render/Railway ki disk ephemeral hoti hai.

Final deliverables

- ☐ Clean GitHub repo (no node_modules, no .env)
- ☐ .env.example (frontend + backend + FastAPI)
- ☐ README with installation steps
- ☐ db/schema.sql
- ☐ API documentation (Postman/Swagger)
- ☐ Screenshots of all pages
- ☐ Demo dataset (sample CSV)
- ☐ Project report + presentation

> **Next step:** Backend Phase 1 (server + DB connection) aur Frontend Phase 1 (setup + routing) se aaj hi shuru karo. Kisi bhi phase ka full code chahiye ho to bolo — main us part ko complete build karke de dunga.